

Wine Reviews Rerun Plan for FT+PPI Mean Estimation

Prepared for Internal Discussion

June 12, 2026

1 Objective of the Rerun

This document provides a complete experimental redesign for rerunning the Wine Reviews mean-estimation experiment in the FT+PPI paper. The goal is not to make Wine Reviews the final main empirical application. Rather, the goal is to use Wine Reviews as a controlled benchmark to validate the full experimental pipeline before migrating the same workflow to more management-relevant mean-estimation tasks and to general M-estimation tasks.

The rerun should satisfy four requirements:

- (i) All label accounting must be clean. If a label is used for ramp-up, validation, early stopping, hyperparameter selection, fine-tuning, or scaling-law fitting, this cost must be charged against the labeled budget.
- (ii) The ramp-up procedure must be part of the deployable estimator, not an oracle preprocessing step using extra labels.
- (iii) The experiment must include stronger inferential baselines, especially PPI++, Cross-PPI, calibrated PPI, and the old Qwen3-Embedding-0.6B + MLP replication baseline.
- (iv) The experiment must be implemented in a modular way so that the same codebase can later be reused for categorical response estimation and general M-estimation.

The key point is that the Wine rerun should be treated as an implementation blueprint. The scientific standard should be high enough that, once the same logic is moved to a better dataset or M-estimation task, the empirical section can be defended without redesigning the methodology.

2 Original Wine Experiment and Main Weaknesses

The previous Wine Reviews experiment used the Wine Enthusiast dataset. After removing duplicated reviews, the empirical analysis contained roughly 119,955 unique reviews. The input was the review text only, and the outcome was the expert rating on an 80–100 scale. The old experiment treated 109,955 reviews as the target population and held out 10,000 additional labeled reviews to estimate the fine-tuning scaling law. The target population mean was approximately 88.441.

The old implementation used Qwen3-Embedding-0.6B as a text encoder. The final hidden-layer embedding was passed into a small multilayer perceptron regression head. Most encoder parameters were frozen; only the final transformer layer and the MLP head were updated. The ramp-up procedure was described in the online appendix: the labeled sample was split into a fine-tuning subset, a validation subset, and a PPI subset. The validation subset was used to estimate the scaling law and was not used for either fine-tuning or final PPI correction.

The old setup was useful as proof of concept, but it has several limitations that must be addressed in the rerun:

- (a) The main scaling-law validation used an extra 10,000 labeled observations. This makes label accounting vulnerable under the limited-human-data framing.
- (b) The ramp-up procedure was placed in the appendix rather than being implemented as part of the main estimator comparison.
- (c) The main figures focused on estimated means and confidence intervals, which made the optimal allocation visually difficult to identify.
- (d) The baselines were too limited. In particular, PPI++, Cross-PPI, calibrated PPI, and a clean replication of the old Qwen3-Embedding-0.6B + MLP pipeline were missing.
- (e) The model architecture looked more like embedding regression than LLM fine-tuning. A rerun should use LoRA or QLoRA on an instruction-tuned causal language model.

3 Overall Structure of the New Wine Empirical Section

The rerun should be organized as follows.

Section	Purpose	Main Outputs
Empirical setup and label accounting	Define data, estimand, labeled budget, unlabeled pool, and models	Data flow diagram and label accounting table
Ramp-up and scaling-law validation	Move ramp-up into the main experiment and test the residual-variance learning curve	Scaling-law curve, leave-one-size-out error, bootstrap intervals
Optimal allocation validation	Compare theoretical allocation, ramp-up allocation, and empirical oracle allocation	Allocation curve, normalized variance curve, regret table
Estimator comparison	Compare sample mean, PPI, PPI++, calibrated PPI, Qwen3-Embedding-0.6B + MLP, LoRA FT+PPI, and Cross-PPI	RMSE, bias, empirical SD, estimated SE, CI length, coverage, sample savings
Diagnostics and robustness	Address collapse, MSE versus variance loss, model scale, Cross-PPI, and external pilot labels	Residual variance, prediction variance, calibration, ablations

4 Data Setup

4.1 Cleaning Rules

Use the same core setting as the original paper:

$$X_i = \text{wine review description}, \quad Y_i = \text{rating points} \in \{80, 81, \dots, 100\}.$$

The cleaning protocol should be fixed before running any experiment:

1. Drop observations with missing **description** or missing **points**.
2. Deduplicate by **description**. If the same description maps to multiple ratings, either retain the first occurrence or use a deterministic rule such as the modal rating. The chosen rule must be documented and fixed.
3. Exclude all structured metadata from the main experiment, including variety, country, price, province, winery, and designation. This maintains comparability with the original experiment and prevents gains from being attributed to metadata rather than textual prediction.
4. Record the finite population size N , finite population mean μ_N , finite population variance σ_Y^2 , the rating histogram, and the token-length distribution under each tokenizer.

The rating distribution should be reported. Since the ratings are concentrated between 80 and 100 and often around the high 80s, this task may naturally make different methods look similar. This is not a reason to hide the limitation. It should be reported as part of the empirical context.

4.2 Finite Population Protocol

Treat the cleaned Wine Reviews dataset as a finite target population:

$$\mathcal{P} = \{(X_i, Y_i)\}_{i=1}^N, \quad \mu_N = \frac{1}{N} \sum_{i=1}^N Y_i.$$

For each replication r and labeled budget B :

1. Draw a labeled set $L_r \subset \mathcal{P}$ of size B by simple random sampling without replacement.
2. Define the unlabeled set as $U_r = \mathcal{P} \setminus L_r$. The experiment may use X_i for $i \in U_r$ but may not use Y_i for model training, model selection, scaling-law fitting, or estimator construction.
3. All methods in the same replication must use the same L_r and U_r . This enables paired method comparisons.

Use the following labeled budgets:

$$B \in \{1000, 3000, 5000, 10000\}.$$

A smaller setting such as $B = 500$ can be used for debugging or appendix robustness, but it should not be part of the main table unless the ramp-up cost is handled carefully.

Recommended replication counts:

- Debug stage: $R = 3$.
- Pilot stage: $R = 10$.
- Full Wine rerun: target $R = 30$; if GPU resources are constrained, use at least $R = 20$ for the main tables.

5 Model Architecture

5.1 Main LoRA / QLoRA Model

The rerun should replace the old embedding-plus-MLP main model with LoRA or QLoRA fine-tuning of an instruction-tuned causal language model.

Recommended model tiers:

Tier	Model	Purpose
Debug	Qwen2.5-1.5B-Instruct or another small Qwen model	Fast pipeline check for losses, PPI estimators, and Hyak jobs
Main	Qwen2.5-7B-Instruct	Main Wine rerun model
Robustness	Llama-3.1-8B-Instruct or Qwen3-4B/8B	Architecture or model-family robustness

The main model should treat the wine rating as a continuous scalar outcome, not as a discrete classification problem. Use a regression head on top of the instruction-tuned model representation, following the original continuous-regression setup but replacing the old embedding encoder with a LoRA/QLoRA-adapted causal language model.

For each input review, use a fixed text template such as:

System: You are an expert wine critic. Given a wine review, predict the expert rating on an 80-100 scale.

User:

Review: {description}

Target:

Continuous rating score on the 80-100 scale.

Let $h_\theta(X_i)$ denote the pooled final hidden representation of this prompted input, for example the final-token representation or another fixed pooling rule chosen before the experiment. The surrogate prediction is:

$$f_\theta(X_i) = w^\top h_\theta(X_i) + b, \quad f_\theta(X_i) \in \mathbb{R}.$$

The model is trained against the observed numerical rating Y_i as a continuous response. Do not normalize over the discrete labels $\{80, \dots, 100\}$, and do not train the main Wine rerun as a classification task.

This design has three advantages. First, it preserves the mean-estimation setting. Second, it stays close to the original Wine experiment’s continuous regression target, making the rerun easier to compare with the current paper. Third, it keeps the prediction object as a scalar $f_\theta(X)$, which is exactly the quantity needed by PPI, PPI++, and Cross-PPI.

5.2 Embedding Baseline

The rerun should retain only the old embedding-regression pipeline as the embedding / ML baseline:

Baseline	Feature Representation	Learner
----------	------------------------	---------

Qwen3-Embedding-0.6B + MLP Frozen or partially tuned embedding Old-paper replication baseline

Do not add other classical ML or embedding baselines to the Wine rerun. Keeping only Qwen3-Embedding-0.6B + MLP makes the comparison focused: the rerun asks whether the new label accounting, ramp-up, and PPI variants improve the old Wine pipeline.

6 LoRA / QLoRA Training Configuration

6.1 Default Configuration

Recommended default settings:

Parameter	Recommended Value
Quantization	4-bit NF4, bfloat16 compute
LoRA rank r	16
LoRA alpha	32
LoRA dropout	0.05
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Max sequence length	256 first; if more than 5% of examples are truncated, use 384
Optimizer	paged_adamw_8bit or AdamW
Learning rate	2×10^{-4}
Learning-rate robustness	1×10^{-4} and 5×10^{-5}
Weight decay	0
Warmup ratio	0.03
Scheduler	cosine
Epochs	3 for $s \geq 1000$; 5 for $s < 1000$
Early stopping	Patience of 2 evaluations
Evaluation frequency	Every 0.25 epoch or every 50 steps
Effective batch size	At least 32 if possible; 64 preferred

6.2 Important Implementation Issue: Variance Loss and Micro-Batches

The residual-variance loss is a batch-level objective:

$$L_{\text{var}} = \frac{1}{q} \sum_{i=1}^q (r_i - \bar{r})^2, \quad r_i = Y_i - f_{\theta}(X_i), \quad \bar{r} = \frac{1}{q} \sum_{i=1}^q r_i.$$

If the micro-batch size is too small, the batch variance estimate will be noisy and unstable. Standard gradient accumulation is not automatically equivalent to computing the variance over the full effective batch, because each micro-batch may already backpropagate a different batch-centered loss.

There are two acceptable implementations:

1. Use a true forward batch size of at least 32 whenever GPU memory allows.

2. Implement custom accumulated loss: forward multiple micro-batches, collect residuals, compute the variance loss over the concatenated effective batch, and only then backpropagate.

A rating-stratified batch sampler is recommended so that each batch contains observations from multiple rating levels. This reduces instability when ratings are concentrated near the population mean.

6.3 Training Losses

The LoRA pipeline should implement two primary continuous-regression losses.

MSE Loss.

$$L_{\text{MSE}} = \frac{1}{q} \sum_{i=1}^q (Y_i - f_{\theta}(X_i))^2.$$

This is included to align with the original MSE baseline.

Residual-Variance Loss.

$$L_{\text{Var}} = \frac{1}{q} \sum_{i=1}^q \left[(Y_i - f_{\theta}(X_i)) - \frac{1}{q} \sum_{j=1}^q (Y_j - f_{\theta}(X_j)) \right]^2.$$

This is the main proposed fine-tuning objective.

As a robustness check, one may also evaluate an anchored version:

$$L_{\text{AnchoredVar}} = L_{\text{Var}} + \lambda \bar{r}^2, \quad \lambda \in \{0.01, 0.05\}.$$

This should not be the main method because it changes the theoretical objective. It is only a diagnostic for identifiability and collapse concerns.

7 Ramp-Up Procedure with Clean Label Accounting

The ramp-up procedure should be moved into the main empirical design. The main result should use target-only label accounting. The old external 10,000-label procedure can be reported only as a separate robustness regime.

7.1 Main Regime: Target-Only Conservative Ramp-Up

Given a labeled budget B , draw a validation set:

$$V_r \subset L_r, \quad |V_r| = v_B.$$

Recommended validation sizes:

Budget B	Validation size v_B	Effective final budget $n_{\text{eff}} = B - v_B$
1000	200	800
3000	300	2700
5000	500	4500

10000 1000 9000

The remaining labeled set is:

$$L'_r = L_r \setminus V_r, \quad n_{\text{eff}} = B - v_B.$$

Construct a nested sequence of fine-tuning sets:

$$T_{s_1} \subset T_{s_2} \subset \cdots \subset T_{s_K} \subset L'_r, \quad |T_{s_\ell}| = s_\ell.$$

Recommended ramp-up grids:

Budget B	Validation size v_B	Fine-tuning size grid s_ℓ
1000	200	50, 100, 150, 250, 400
3000	300	100, 200, 400, 700, 1000, 1500
5000	500	100, 200, 400, 700, 1000, 1500, 2500
10000	1000	100, 200, 400, 700, 1000, 1500, 2500, 4000

For each s_ℓ :

1. Fine-tune a LoRA model on T_{s_ℓ} .
2. Evaluate validation residual variance on V_r :

$$\widehat{\nu}(s_\ell) = \widehat{\text{Var}}_{V_r} \{Y - f^{(s_\ell)}(X)\}.$$

3. Fit the residual-variance scaling law:

$$\widehat{\nu}(s) = \hat{a}s^{-\hat{\alpha}} + \hat{b}.$$

4. Choose the allocation by solving:

$$\hat{s}^* = \arg \min_{0 < s < n_{\text{eff}}} \frac{\hat{a}s^{-\hat{\alpha}} + \hat{b}}{n_{\text{eff}} - s}.$$

5. Choose the nearest feasible grid point or continue ramp-up if the predicted optimum exceeds the current largest training size.
6. Train the final model on $T_{\hat{s}}$ and use the correction set

$$C_r = L'_r \setminus T_{\hat{s}}$$

for PPI correction.

The validation set V_r must not be used in final PPI correction, because it has been used for scaling-law fitting, checkpoint selection, and model selection. This conservative protocol charges the validation labels to the method. It is stricter than the original paper and therefore more defensible.

7.2 Robustness Regime: External Pilot Labels Available

As a separate practical regime, replicate the spirit of the original experiment:

1. Set aside 10,000 labeled reviews as an external pilot set H .
2. Use H to estimate scaling-law parameters, tune the prompt format, and choose hyperparameters.
3. Use the remaining reviews as the target population.
4. For each target labeled budget B , do not charge H against the current-task budget, but clearly report this as an external-pilot setting.

This should not be the main result. It is a practical scenario corresponding to firms that have historical expert ratings or survey archives.

7.3 Optional Robustness: No-Waste Cross-Validated Ramp-Up

If compute resources allow, add an appendix version that does not permanently set aside a validation set. For each candidate fine-tuning size s , estimate residual variance by cross-validation inside T_s , then train the final model on all of T_s and reserve $L_r \setminus T_s$ for PPI correction.

This version saves labels but is much more expensive because it requires multiple fine-tuning runs per candidate size. It should be optional.

8 Scaling-Law Validation

The scaling-law section should no longer rely only on in-sample R^2 . It should validate both the curve and the allocation decision.

8.1 Scaling Curve

For $B = 10000$, plot:

$$s \mapsto \hat{\nu}(s) = \widehat{\text{Var}}_{V_r} \{Y - f^{(s)}(X)\}.$$

The figure should include:

1. The fitted power-law curve in linear space.
2. A log-log plot of $\hat{\nu}(s) - \hat{b}$ against s .
3. Bootstrap bands across replications.
4. A constant-predictor baseline $\widehat{\text{Var}}(Y)$.

The main panel can focus on LoRA-Var, while appendix panels show LoRA-MSE.

8.2 Leave-One-Size-Out Prediction Error

For each fine-tuning size s_ℓ in the grid:

1. Remove the point $(s_\ell, \hat{\nu}(s_\ell))$.
2. Fit $\hat{a}$, $\hat{\alpha}$, and $\hat{b}$ using the remaining sizes.
3. Predict the held-out residual variance.
4. Report the normalized prediction error:

$$\text{LOSO Error} = \frac{|\hat{\nu}(s_\ell) - \hat{\nu}_{-\ell}(s_\ell)|}{\hat{\nu}(s_\ell)}.$$

This directly addresses the concern that a smooth parametric curve may overfit noisy small-sample observations.

8.3 Allocation Sensitivity

Bootstrap the fitted scaling-law parameters and report:

- confidence intervals for $\hat{a}$, $\hat{\alpha}$, and $\hat{b}$;
- confidence intervals for $\hat{s}^*/B$;
- allocation regret:

$$\text{Regret}(\hat{s}) = \frac{V(\hat{s}) - V(s^{\text{oracle}})}{V(s^{\text{oracle}})}.$$

The allocation decision matters more than the exact parameter estimates. If $\hat{s}^*$ is not exact but regret is low, the method is still practically useful.

9 Optimal Allocation Validation

The original allocation figure should be replaced. Instead of plotting estimated means and confidence intervals, plot normalized estimator variance or normalized CI length directly.

9.1 Oracle Allocation Grid

For $B = 10000$, evaluate the following allocation ratios:

$$\frac{s}{B} \in \{0, 0.025, 0.05, 0.075, 0.10, 0.125, 0.15, 0.20, 0.30, 0.40, 0.50, 0.70, 0.90\}.$$

If $s < 50$, treat the method as no-FT or PPI-only rather than training an unstable model.

For each allocation s :

1. Train $f^{(s)}$ using the s fine-tuning labels.
2. Use $B - s$ labels for PPI correction.
3. Use all unlabeled observations in U_r for the prediction term.

4. Repeat over replications and compute estimator distribution, empirical variance, confidence interval length, and coverage.

This oracle grid is not a deployable method. It is used to validate the theory and to show the shape of the allocation tradeoff.

9.2 Recommended Figure

The new allocation figure should have three panels.

Panel A: Residual-Variance Learning Curve.

$$\hat{\nu}(s) = \widehat{\text{Var}}\{Y - f^{(s)}(X)\}.$$

Panel B: Normalized Estimator Variance or CI Width. Estimate:

$$\hat{V}_{\text{PPI}}(s) = \frac{\hat{\nu}_C(s)}{B-s} + \frac{\hat{\tau}_U(s)}{|U_r|}, \quad \hat{\tau}_U(s) = \widehat{\text{Var}}_{U_r}\{f^{(s)}(X)\}.$$

Plot:

$$\frac{\hat{V}_{\text{PPI}}(s)}{\widehat{V}_{\text{SampleMean}}}.$$

This is much clearer than plotting estimated means.

Panel C: Efficiency Gain.

$$R^2(s) - \frac{s}{B}, \quad R^2(s) = 1 - \frac{\hat{\nu}(s)}{\widehat{\sigma}_Y^2}.$$

This panel directly corresponds to the theoretical criterion for outperforming the sample mean.

9.3 Allocation Validation Table

Report:

Method	$\hat{s}^*/B$	s^{oracle}/B	Absolute gap	Regret	Normalized CI length
LoRA-Var					
LoRA-MSE					

MSE does not have a theoretically privileged downstream loss, but its residual-variance curve can still be fitted and used to compute an allocation benchmark.

10 Estimator Comparison

The main comparison should be organized by estimator family. All comparisons should use paired replications.

10.1 Label-Only Baseline

Sample Mean.

$$\hat{\mu}_{\text{mean}} = \frac{1}{B} \sum_{i \in L_r} Y_i.$$

This uses all B labels and is the most important baseline.

10.2 Fixed-Predictor Rectification Baselines

Zero-Shot LLM + PPI. Use the base LLM without any label-based fine-tuning. All B labels are used for correction.

Zero-Shot LLM + PPI++ / Linear Calibrated PPI. For mean estimation, use the control-variate form:

$$\hat{\mu}_\lambda = \bar{Y}_C + \hat{\lambda}(\bar{f}_U - \bar{f}_C).$$

A finite-sample estimate is:

$$\hat{\lambda} = \frac{\widehat{\text{Cov}}_C(Y, f)}{\widehat{\text{Var}}_C(f)} \cdot \frac{|U|}{|U| + |C|}.$$

To avoid overfitting λ , use a 5-fold cross-fitted λ : for each correction fold, estimate λ using the other folds.

10.3 Qwen3 Embedding Replication Baseline

Include:

1. Qwen3-Embedding-0.6B + MLP + PPI.
2. Qwen3-Embedding-0.6B + MLP + PPI++.

For fair comparison, report two allocation versions when feasible:

- same allocation as the LoRA-Var ramp-up rule;
- own oracle allocation over the grid, reported as an upper-bound baseline.

10.4 Fine-Tuning Only

Report LoRA-FT-only methods using MSE and Var losses. These use all B labels for fine-tuning and estimate the mean by averaging $f(X)$ over the population. These estimators are generally biased. Therefore, do not report them as valid inferential methods with PPI confidence intervals. Report RMSE and bias only.

10.5 Split FT+PPI

Main methods:

1. LoRA-MSE + PPI.
2. LoRA-Var + PPI.

Each should be reported in two versions:

1. Oracle allocation: grid-search best allocation, reported as an upper bound.
2. Ramp-up allocation: deployable main result with all ramp-up labels counted.

10.6 Split FT+PPI++

Add:

1. LoRA-MSE + PPI++.
2. LoRA-Var + PPI++.

This is important. If LoRA-Var still improves after PPI++ calibration, then the contribution is not merely due to using a weak rectification baseline.

10.7 Cross-PPI

Use $K = 5$ folds. Split L_r into $I_1, \dots, I_K$. For fold k , train f_{-k} without using the labels in I_k . Estimate:

$$\hat{\mu}_{\text{CrossPPI}} = \frac{1}{K} \sum_{k=1}^K \frac{1}{|U_r|} \sum_{u \in U_r} f_{-k}(X_u) + \frac{1}{B} \sum_{k=1}^K \sum_{i \in I_k} (Y_i - f_{-k}(X_i)).$$

Run at least:

1. Cross-PPI-MSE.
2. Cross-PPI-Var.

Cross-PPI is not just another baseline. It directly addresses the question of whether sample splitting is necessary. The strongest result would be that Cross-PPI-Var improves over Cross-PPI-MSE, showing that the variance-aligned objective complements cross-fitting.

10.8 Main Comparison Table

For each budget B , report:

Method	RMSE	Bias	Emp. SD	Est. SE	CI Length	Coverage	Sample Savings
Sample Mean							
PPI							
PPI++							
Qwen3-Embedding-0.6B +							
MLP + PPI++							
LoRA-MSE + PPI++							
LoRA-Var + PPI++							
Cross-PPI-MSE							
Cross-PPI-Var							

Definitions:

$$\text{RMSE} = \sqrt{\frac{1}{R} \sum_{r=1}^R (\hat{\mu}_r - \mu_N)^2},$$

$$\text{Bias} = \frac{1}{R} \sum_{r=1}^R (\hat{\mu}_r - \mu_N),$$

$$\text{SampleSavings} = 1 - \frac{\widehat{\text{Var}}(\hat{\mu}_{\text{method}})}{\widehat{\text{Var}}(\hat{\mu}_{\text{SampleMean}})}.$$

All method comparisons should report paired standard errors. For a metric M , define:

$$d_r = M_{r,A} - M_{r,B}.$$

Report $\bar{d}$ and its paired standard error rather than only reporting unpaired standard errors for each estimator.

11 Confidence Intervals and Variance Estimation

11.1 Vanilla PPI

For a fixed predictor f :

$$\hat{\mu}_{\text{PPI}} = \bar{f}_U + \overline{Y - f(X)}_C.$$

Estimated variance:

$$\widehat{\text{Var}}(\hat{\mu}) = \frac{\widehat{\text{Var}}_C(Y - f(X))}{|C|} + \frac{\widehat{\text{Var}}_U(f(X))}{|U|}.$$

The 95% confidence interval is:

$$\hat{\mu} \pm 1.96 \sqrt{\widehat{\text{Var}}(\hat{\mu})}.$$

11.2 PPI++ / Calibrated PPI

For:

$$\hat{\mu}_\lambda = \bar{Y}_C + \hat{\lambda}(\bar{f}_U - \bar{f}_C),$$

use:

$$\widehat{\text{Var}}(\hat{\mu}_\lambda) = \frac{\widehat{\text{Var}}_C(Y - \hat{\lambda}f(X))}{|C|} + \frac{\hat{\lambda}^2 \widehat{\text{Var}}_U(f(X))}{|U|}.$$

If $\hat{\lambda}$ is cross-fitted, compute residuals fold by fold using the corresponding out-of-fold $\hat{\lambda}$ and then aggregate the variance.

11.3 Coverage

Coverage is evaluated against the finite population mean μ_N :

$$\mathbf{1}\{\mu_N \in CI_r\}.$$

The main text can use the standard PPI variance estimator. A finite-population correction can be included as an appendix robustness check.

12 Diagnostics and Robustness

Every trained model should save the following diagnostics on validation, correction, and unlabeled sets whenever labels are available.

12.1 Prediction Diagnostics

Report:

- residual mean $\overline{Y - f(X)}$;
- residual variance $\widehat{\text{Var}}(Y - f(X))$;
- MSE;
- MAE;
- prediction variance $\widehat{\text{Var}}(f(X))$;
- Pearson and Spearman correlations between Y and $f(X)$;
- calibration intercept and slope from $Y = \alpha + \beta f(X) + \varepsilon$.

12.2 Collapse Diagnostics

To address representation-collapse concerns, report:

1. whether $\widehat{\text{Var}}(f(X))$ is close to zero;
2. whether residual variance is lower than the constant-predictor baseline;
3. the histogram and quantiles of continuous predictions $f(X)$;
4. the share of predictions outside the observed rating range $[80, 100]$ before any optional clipping.

If Var loss outputs nearly constant predictions without reducing residual variance, that is a failure mode. If Var loss preserves nontrivial prediction variance and lowers residual variance, it directly addresses the collapse criticism.

12.3 MSE versus Var Mechanism Figure

Add an appendix figure with:

- x -axis: fine-tuning size s ;
- y -axis: validation residual variance;
- curves: MSE, Var, and constant predictor.

A second panel can show validation MSE. The goal is to show that MSE may perform well on prediction error but does not necessarily minimize the residual variance that controls PPI efficiency.

13 Hyak Execution Plan

UW Hyak uses Slurm, so the code should be designed around Slurm job arrays and checkpointed training.

13.1 Recommended Code Structure

```
src/  
  data/  
    build_wine.py  
    split_protocol.py  
  models/  
    regression_model.py  
    lora_trainer.py  
    embedding_baselines.py  
  losses/  
    mse_regression_loss.py  
    residual_variance_loss.py  
  estimators/  
    mean.py  
    ppi.py  
    ppi_plus.py  
    cross_ppi.py  
  allocation/  
    scaling_fit.py  
    ramp_up.py  
    oracle_grid.py  
  experiments/  
    exp00_debug.py  
    exp01_scaling_law.py  
    exp02_allocation.py  
    exp03_estimator_comparison.py  
    exp04_cross_ppi.py  
  analysis/  
    make_tables.py  
    make_figures.py  
  configs/  
    wine_debug.yaml  
    wine_full_qwen25_7b.yaml  
    wine_crossppi.yaml  
  slurm/  
    run_lora_array.sbatch  
    run_eval_array.sbatch
```

Training and estimation should be separated. Each trained model should produce predictions. PPI, PPI++, and Cross-PPI should be recomputed offline from saved prediction files.

13.2 Prediction Cache Schema

Save predictions in a long-format Parquet file with columns such as:

```
run_id  
replication_id  
budget_B  
method  
loss  
model_name  
allocation_s  
fold_id
```

```

sample_id
split_role # train / validation / correction / unlabeled
y_true # missing or masked for unlabeled in estimator code
pred_mean
pred_raw
pred_var_optional
pred_clipped_optional

```

This schema is designed for the Wine continuous-regression rerun. It can later be extended with probability columns if a separate categorical response task is added.

13.3 Slurm Job Array Design

Each LoRA training run should correspond to a Slurm array task:

```
TASK = (model, loss, budget, replication, allocation_s, fold_id)
```

For split FT+PPI, use `fold_id = 0`. For Cross-PPI, use `fold_id = 1, ..., K`.

14 Execution Phases

14.1 Phase 0: Non-LLM Unit Tests

Use synthetic data to test:

- PPI unbiasedness;
- PPI++ variance behavior;
- Cross-PPI fold logic;
- confidence interval coverage under simple data-generating processes.

No GPU is needed.

14.2 Phase 1: Wine Embedding Baseline

Run:

- sample mean;
- Qwen3-Embedding-0.6B + MLP old-paper replication.

The purpose is to validate the split protocol, label accounting, estimator code, and table generation.

14.3 Phase 2: LoRA Debug

Use Qwen2.5-1.5B-Instruct.

- Budgets: $B = 1000$ and $B = 5000$.
- Replications: $R = 3$.

- Losses: MSE, Var.
- Allocations: $s/B \in \{0.1, 0.2\}$.

Check training stability, residual variance, prediction variance, PPI coverage, GPU memory, and runtime.

14.4 Phase 3: Scaling Law and Allocation

Use Qwen2.5-7B-Instruct.

- Budget: $B = 10000$.
- Replications: start with $R = 10$, then increase to $R = 20$ or $R = 30$ if stable.
- Allocation grid:

$$s/B \in \{0.025, 0.05, 0.075, 0.10, 0.125, 0.15, 0.20, 0.30, 0.40, 0.50\}.$$

Run larger allocation ratios such as 0.70 and 0.90 only if needed to display the full U-shape.

14.5 Phase 4: Estimator Comparison

Use Qwen2.5-7B-Instruct.

- Budgets: $B \in \{1000, 3000, 5000, 10000\}$.
- Replications: at least $R = 20$, target $R = 30$.
- Methods: sample mean, PPI, PPI++, Qwen3-Embedding-0.6B + MLP + PPI/PPI++, LoRA-MSE + PPI/PPI++, LoRA-Var + PPI/PPI++, ramp-up allocation, oracle allocation upper bounds.

14.6 Phase 5: Cross-PPI

Run Cross-PPI after the split-pipeline is stable.

- Folds: $K = 5$.
- Budgets: $B = 1000$ and $B = 5000$ first; add $B = 10000$ if resources allow.
- Replications: start with $R = 10$.
- Losses: MSE, Var.

Cross-PPI is about five times more expensive than a single split run, so it should not be run at full scale before the rest of the pipeline is stable.

15 Draft Structure for the New Empirical Section

15.1 Empirical Setup

Emphasize that Wine Reviews is a mean-estimation benchmark. Do not oversell it as a full human-behavior simulation task. The task is estimating average expert ratings from textual expert evaluations.

15.2 Data-Driven Ramp-Up and Scaling-Law Diagnostics

State that the ramp-up procedure is implemented as part of the estimator, not as an oracle pre-processing step. Validation labels are counted. The section should report in-sample fit, leave-one-size-out prediction error, and allocation regret.

15.3 Optimal Allocation Validation

Show the U-shaped allocation curve using normalized estimator variance or normalized CI width. Report the gap between the scaling-law allocation and empirical oracle allocation.

15.4 Comparison with Baselines

Compare against PPI++, Cross-PPI, the Qwen3-Embedding-0.6B + MLP replication baseline, and standard continuous fine-tuning losses. Report coverage and CI length in addition to RMSE.

15.5 Robustness

Include collapse diagnostics, external-pilot labels, model-size robustness, anchored variance loss, and Cross-PPI with variance loss.

16 Design Errors to Avoid

1. Do not use an extra 10,000 labels in the main method while sample mean does not receive them.
2. Do not use validation labels for scaling-law fitting and then reuse them for PPI correction.
3. Do not only plot estimated means against allocation ratios. Plot estimator variance, CI length, and normalized variance ratio.
4. Do not compare only against vanilla PPI. PPI++ and Cross-PPI must appear.
5. Do not report only unpaired standard errors. Use paired comparisons across replications.
6. Do not compute variance loss on tiny micro-batches unless a correct accumulated-loss implementation is used.
7. Do not present FT-only as a valid inferential estimator. It may be reported for RMSE and bias, but not for valid PPI confidence intervals.

17 Minimum Viable Rerun and Full Rerun

17.1 Minimum Viable Rerun

Used to confirm the Hyak pipeline and method implementation:

- Model: Qwen2.5-1.5B-Instruct.
- Budgets: $B = 1000$ and $B = 5000$.
- Replications: $R = 3$.

- Losses: MSE, Var.
- Methods: sample mean, PPI, PPI++, split FT+PPI, Cross-PPI.
- Allocations: $s/B \in \{0.1, 0.2\}$.
- Diagnostics: residual variance, prediction variance, RMSE, coverage.

17.2 Full Wine Rerun

Used for a serious appendix or internal validation:

- Model: Qwen2.5-7B-Instruct.
- Budgets: $B \in \{1000, 3000, 5000, 10000\}$.
- Replications: at least $R = 20$, target $R = 30$.
- Losses: MSE, Var.
- Allocation validation: $B = 10000$ grid over s/B .
- Main estimator comparison: ramp-up selected allocation, oracle allocation upper bounds, PPI and PPI++.
- Cross-PPI: $B = 1000$ and $B = 5000$, $K = 5$, $R = 10$; add $B = 10000$ if resources allow.
- Baselines: sample mean, PPI-only, PPI++, old Qwen3-Embedding-0.6B + MLP replication.

18 Codex Implementation Prompt

The following prompt can be given directly to Codex.

Implement the Wine Reviews rerun pipeline for FT+PPI mean estimation.

Requirements:

1. Clean Wine Reviews data and construct a finite population protocol.
2. Implement reproducible labeled/unlabeled splits for budgets B in {1000,3000,5000,10000}.
3. Implement sample mean, vanilla PPI, PPI++ with cross-fitted lambda, and Cross-PPI estimators.
4. Implement continuous regression prediction for ratings 80-100 using a scalar regression head on the LoRA/QLoRA model representation.
5. Implement LoRA/QLoRA fine-tuning with MSE loss and residual-variance loss.
6. Implement conservative ramp-up with validation labels counted and excluded from PPI correction.
7. Implement scaling-law fitting and allocation selection.
8. Implement oracle allocation grid for B=10000.
9. Implement the frozen embedding baseline: Qwen3-Embedding-0.6B+MLP.
10. Save all predictions and estimator outputs in parquet/JSON so estimators can be recomputed without retraining.
11. Add leakage checks: no correction label may be used in training, validation, scaling-law fitting, or checkpoint selection for the same split model.
12. Produce tables for RMSE, bias, empirical SD, estimated SE, CI length, coverage, sample savings, and paired differences.

13. Produce figures for scaling law, allocation variance curve, efficiency gain $R^2(s)-s/B$, and collapse diagnostics.

Minimum leakage checks:

```
assert train_ids.isdisjoint(correction_ids)
assert validation_ids.isdisjoint(correction_ids)
assert prompt_demo_ids.isdisjoint(correction_ids) # if prompt baselines are used
# For Cross-PPI fold k:
assert fold_k_ids.isdisjoint(train_ids_for_f_minus_k)
```

19 Success Criteria

The Wine rerun should be considered successful if it establishes the following points:

1. The residual-variance scaling law is empirically stable, not only in terms of R^2 but also in leave-one-size-out prediction and allocation regret.
2. The ramp-up selected allocation is close to the empirical oracle allocation, or at least has low regret.
3. The variance loss reduces residual variance relative to MSE.
4. The advantage of variance loss remains after PPI++ or calibrated PPI is applied.
5. Cross-PPI improves label reuse, but Cross-PPI-Var further improves over Cross-PPI-MSE, showing that variance-aligned fine-tuning and cross-fitting are complementary.
6. Confidence interval coverage remains valid.
7. Collapse diagnostics show nontrivial prediction variance and lower residual variance than the constant-predictor baseline.

If these conditions are met on Wine Reviews, the experimental pipeline is ready to be transferred to survey mean estimation, categorical response estimation, and M-estimation tasks.